

ECOCIVIX AI

08 — API Contract

TEAM VELOCITY • HackYatra × EDM 3.0 • PRE-HACKATHON PLANNING

Purpose: Define the stable interface between the React frontend and Express backend. The contract prevents frontend/backend integration drift.

Status: PRE-HACKATHON DRAFT / PLANNING SPECIFICATION

Approval authority: Garv Nain

Contract Principles

- API shapes must be explicit. Frontend must not invent response fields. Backend must not silently rename fields. Breaking changes require coordinated approval.

Authentication

- Use Supabase Auth for identity. Backend verifies authenticated sessions/tokens and enforces role-based authorization.

Issue Endpoints — Planning Baseline

- POST /api/issues — create issue.
- GET /api/issues — list/filter issues according to permissions.
- GET /api/issues/:id — issue details.
- PATCH /api/issues/:id — controlled issue updates.
- DELETE should only exist if deletion is a real product requirement and authorization rules are explicit.

AI Endpoint — Planning Baseline

- POST /api/issues/:id/analyze or an equivalent server-side analysis route.
- Input: issue context permitted by the product.
- Output: validated AI analysis matching the finalized schema.

Assignment

- POST/PATCH issue assignment endpoint should update the responsible department/operator according to authorization rules.

Updates

- POST /api/issues/:id/updates — create a timeline/progress update where permitted.

Analytics

- GET endpoints can expose aggregated metrics for dashboards/community intelligence. Aggregates must be computed from real data.

Response Shape

- Use a consistent success/error envelope if adopted by the team.
- Document field names, types, nullable behavior and enum values before implementation.

Status Codes

- 200/201 for successful reads/creates; 400 for invalid input; 401 for unauthenticated; 403 for unauthorized; 404 for missing resources; 409 for state conflicts where appropriate; 429 for rate limiting; 500/502/503 for server/provider failures as appropriate.

Validation

- Validate request body, path parameters, query parameters and permissions server-side.
- Never trust client-provided role, ownership or status transition authority.

Issue Lifecycle

- OPEN → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED; REJECTED only if approved.
- Backend must enforce permitted transitions.

AI Contract

- Required fields and allowed values are finalized jointly with Document 05. AI output must be validated before persistence.

Versioning

- For a 24-hour MVP, a single /api version is acceptable if the contract is stable. If breaking changes become necessary, coordinate frontend/backend before changing it.

Acceptance Criteria

- Frontend and backend can integrate without guessing; authentication/authorization is explicit; error states are predictable; AI output schema is stable; lifecycle transitions are enforced.